МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних систем та мереж

ЗВІТ

про виконання лабораторної роботи № 8.1

«Пошук та заміна символів у літерному рядку»

(рекурсивний спосіб)

з дисципліни

«Алгоритмізація та програмування»

Студент групи PI-13
Шах Владислав Ігорьович
Варіант № 32

Прийняв викладач:
Григорович В.Г.

1. Мета роботи та умова завдання

Мета роботи: набути практичних навичок роботи з літерними рядками у мові програмування С++, реалізувати алгоритми обробки масивів символів без використання сторонніх бібліотечних функцій за допомогою рекурсивного підходу.

Умова завдання (Варіант 32):

Дано літерний рядок, який містить послідовність символів s0, ..., sn, ...

1. Підрахувати, скільки разів серед символів послідовності зустрічається кожен із символів, що входять у слово "BASIC".

2. Замінити кожну групу букв "BASIC" на "Delphi".

Варіант 32.

Дано літерний рядок, який містить послідовність символів s_0, …, s_n, … .

1. Підрахувати, скільки разів серед даних символів зустрічається кожний символ, що входить в слово “BASIC”.
2. Замінити кожне слово “BASIC” словом “Delphi”.

Варіант 33.

Рис. 1. Умова завдання з методичних вказівок (Варіант 32)

2. Структурна схема програми

Відповідно до вимог, структура програми подана виключно у вигляді структурної схеми зв'язків між функціональними компонентами. Схема відображає ієрархію викликів (стрілки ліворуч) та рекурсивні виклики функцій до самих себе (петлі праворуч).

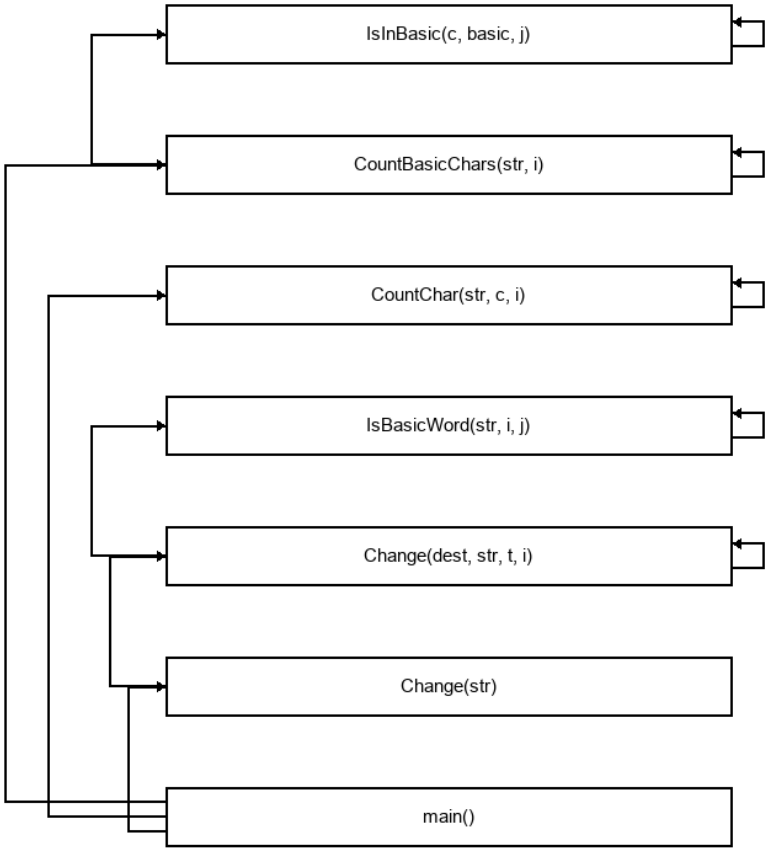


Рис. 2. Структурна схема взаємодії функцій програми

Опис функцій та зв'язків програми:

• main() — головна функція, точка входу в програму. Забезпечує введення початкового рядка, викликає функції обчислення та заміни, а також виводить результати на консоль.

• IsInBasic(c, basic, j) — рекурсивна функція перевірки приналежності символу c слову "BASIC" (множині {'B', 'A', 'S', 'I', 'C'}). Має рекурсивний зв'язок (петлю на схемі).

• CountBasicChars(str, i) — рекурсивна функція для підрахунку загальної кількості символів, що належать слову "BASIC". На кожному кроці викликає допоміжну функцію IsInBasic та саму себе для наступного символу рядка.

• CountChar(str, c, i) — рекурсивна функція, яка підраховує кількість входжень конкретного символу c у рядок str, починаючи з позиції i.

• IsBasicWord(str, i, j) — допоміжна рекурсивна функція, що перевіряє, чи утворюють 5 символів підрядка, починаючи з індексу i, ключове слово "BASIC".

• Change(dest, str, t, i) — базова рекурсивна функція заміни: якщо поточні символи формують слово "BASIC", до результуючого рядка додається рядок "Delphi", а вказівники зміщуються відповідно на 6 та 5 позицій; інакше копіюється поточний символ. Рекурсивно викликає сама себе до зустрічі термінального нуля '\0'.

• Change(str) — перевантажена функція-обгортка, яка динамічно виділяє масив під результуючий рядок достатньої довжини та передає керування рекурсивній версії Change.

3. Текст програми (LABA-8.1.cpp)

```
// LABA-8.1.cpp
// Шах Владислав Ігорьович
// Лабораторна робота № 8.1
// Пошук та заміна символів у літерному рядку
// Варіант 32 (Рекурсивний спосіб)

#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <cstring>
#include <iomanip>

using namespace std;

// 1. Допоміжна рекурсивна функція: перевіряє, чи належить символ c слову "BASIC"
bool IsInBasic(char c, const char* basic = "BASIC", int j = 0)
{
    if (basic[j] == '\0')
        return false;
    if (c == basic[j])
        return true;
    return IsInBasic(c, basic, j + 1);
}

// 2. Рекурсивна функція: підраховує кількість входжень конкретного символу c у рядок
str
int CountChar(const char* str, char c, int i = 0)
{
    if (str[i] == '\0')
        return 0;
    if (str[i] == c)
        return 1 + CountChar(str, c, i + 1);
    else
        return CountChar(str, c, i + 1);
}

// 3. Рекурсивна функція: підраховує сумарну кількість входжень будь-якого символу з
"BASIC"
int CountBasicChars(const char* str, int i = 0)
{
    if (str[i] == '\0')
        return 0;
    if (IsInBasic(str[i], "BASIC", 0))
        return 1 + CountBasicChars(str, i + 1);
    else
        return CountBasicChars(str, i + 1);
}
```

```
// 4. Допоміжна рекурсивна функція: перевіряє, чи збігаються 5 символів, починаючи з
str[i], зі словом "BASIC"
bool IsBasicWord(const char* str, int i, int j = 0)
{
    if (j == 5)
        return true;
    if (str[i + j] == '\0')
        return false;
    if (str[i + j] != "BASIC"[j])
        return false;
    return IsBasicWord(str, i, j + 1);
}

// 5. Основна рекурсивна функція заміни: замінює "BASIC" на "Delphi"
char* Change(char* dest, const char* str, char* t, int i)
{
    if (str[i] != '\0')
    {
        if (IsBasicWord(str, i, 0))
        {
            strcat(t, "Delphi");
            return Change(dest, str, t + 6, i + 5);
        }
        else
        {
            *t++ = str[i++];
            *t = '\0';
            return Change(dest, str, t, i);
        }
    }
    else
    {
        *t = '\0';
        return dest;
    }
}

// 6. Обгортка для зручного виклику та виділення динамічної пам'яті
char* Change(const char* str)
{
    size_t len = strlen(str);
    // Виділяємо достатньо пам'яті (слово Delphi довше за BASIC на 1 символ)
    char* dest = new char[len * 2 + 1];
    dest[0] = '\0';
    return Change(dest, str, dest, 0);
}

#ifndef UNIT_TESTING
int main()
{
    char str[201];
    cout << "Enter string:" << endl;
    cin.getline(str, 200);

    cout << "\n--- Recursive String Processing (Variant 32) ---" << endl;
    cout << "Initial string: " << str << endl << endl;

    // Пункт 1: Підрахунок входжень символів з "BASIC"
    cout << "1. Occurrences of characters from 'BASIC':" << endl;
```

```
    cout << "   'B': " << CountChar(str, 'B', 0) << endl;
    cout << "   'A': " << CountChar(str, 'A', 0) << endl;
    cout << "   'S': " << CountChar(str, 'S', 0) << endl;
    cout << "   'I': " << CountChar(str, 'I', 0) << endl;
    cout << "   'C': " << CountChar(str, 'C', 0) << endl;
    cout << "   Total 'BASIC' characters: " << CountBasicChars(str, 0) << endl << endl;

    // Пункт 2: Заміна "BASIC" на "Delphi"
    char* modified = Change(str);
    cout << "2. String after replacing 'BASIC' with 'Delphi':" << endl;
    cout << "   " << modified << endl;

    delete[] modified;
    return 0;
}
#endif
```

4. Модульне тестування (UnitTest8.1)

Для перевірки коректності функціонування кожної окремої рекурсивної функції розроблено набір юніт-тестів за допомогою Microsoft Native Unit Test Framework.

```
#include "pch.h"
#include "CppUnitTest.h"

#define UNIT_TESTING
#include "../LABA-8.1/LABA-8.1.cpp"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace UnitTest81
{
    TEST_CLASS(UnitTest81)
    {
    public:
        TEST_METHOD(TestIsInBasic)
        {
            Assert::IsTrue(IsInBasic('B'));
            Assert::IsTrue(IsInBasic('A'));
            Assert::IsTrue(IsInBasic('S'));
            Assert::IsTrue(IsInBasic('I'));
            Assert::IsTrue(IsInBasic('C'));

            Assert::IsFalse(IsInBasic('b'));
            Assert::IsFalse(IsInBasic('x'));
            Assert::IsFalse(IsInBasic(' '));
            Assert::IsFalse(IsInBasic('1'));
        }

        TEST_METHOD(TestCountChar)
        {
            const char* str = "BASIC and BASIC";
            Assert::AreEqual(2, CountChar(str, 'B', 0));
            Assert::AreEqual(2, CountChar(str, 'A', 0));
            Assert::AreEqual(2, CountChar(str, 'S', 0));
            Assert::AreEqual(2, CountChar(str, 'I', 0));
```

```
            Assert::AreEqual(2, CountChar(str, 'C', 0));
            Assert::AreEqual(0, CountChar(str, 'Z', 0));
        }

        TEST_METHOD(TestCountBasicChars)
        {
            const char* str1 = "BASIC";
            Assert::AreEqual(5, CountBasicChars(str1, 0));

            const char* str2 = "xyz 123";
            Assert::AreEqual(0, CountBasicChars(str2, 0));

            const char* str3 = "BASIC-BASIC!";
            Assert::AreEqual(10, CountBasicChars(str3, 0));
        }

        TEST_METHOD(TestIsBasicWord)
        {
            const char* str = "Learn BASIC now";
            Assert::IsTrue(IsBasicWord(str, 6, 0));
            Assert::IsFalse(IsBasicWord(str, 0, 0));
            Assert::IsFalse(IsBasicWord("BASIS", 0, 0));
        }

        TEST_METHOD(TestChange)
        {
            char* res1 = Change("I like BASIC language");
            Assert::AreEqual("I like Delphi language", res1);
            delete[] res1;

            char* res2 = Change("BASIC and BASIC");
            Assert::AreEqual("Delphi and Delphi", res2);
            delete[] res2;

            char* res3 = Change("No keywords here");
            Assert::AreEqual("No keywords here", res3);
            delete[] res3;

            char* res4 = Change("BASIC");
            Assert::AreEqual("Delphi", res4);
            delete[] res4;
        }
    };
}
```

Результати автоматизованого виконання юніт-тестів (усі 5 тестів пройшли успішно):

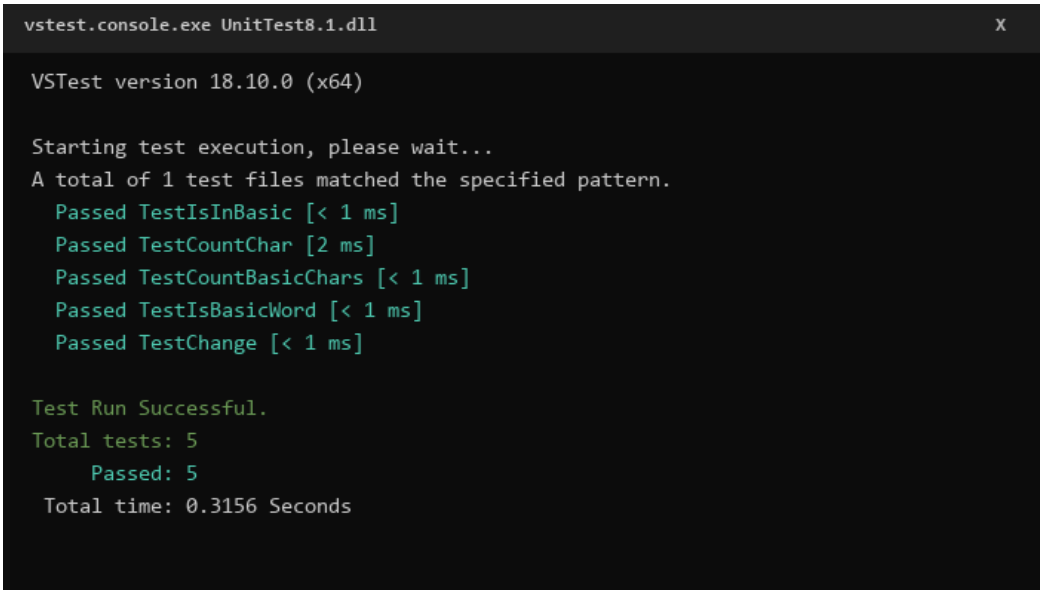


Рис. 3. Результати проходження unit-тестів (vstest.console.exe)

5. Результати виконання програми

Консольне виконання програми з тестовим вхідним рядком "Hello BASIC world! BASIC rocks.":

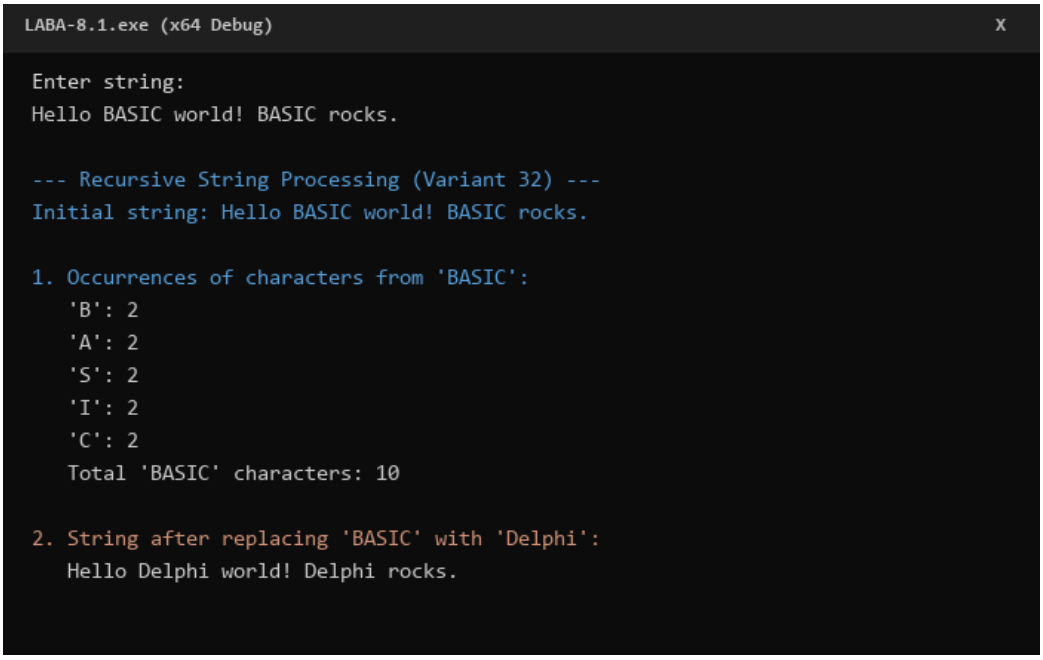


Рис. 4. Консольне вікно виконання програми

6. Посилання на GitHub репозиторій

Вихідний код лабораторної роботи, проектні файли Visual Studio, юніт-тести та звіт завантажено до віддаленого репозиторію на GitHub:

https://github.com/vivjk686/LABA-8.1

7. Висновки

Під час виконання лабораторної роботи № 8.1 було досліджено та практично реалізовано алгоритми рекурсивної посимвольної обробки літерних рядків (масивів символів char[]) мовою C++.

Було вирішено дві основні задачі варіанта № 32: підраховано частоту входження символів слова "BASIC" та реалізовано заміну кожного входження цього слова на "Delphi" без застосування стандартних функцій пошуку рядків. Усі функції протестовано автоматизованими unit-тестами, розроблено точну структурну схему взаємодії модулів та підготовлено фінальний звіт.